

11.1 A 1nm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications

Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Nahsung Kim, Yongkee Kwon, Kornijuk Vladimir, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Jaewook Lee, Donguc Ko, Younggun Jun, Keewon Cho, Ilwoong Kim, Choungki Song, Chunseok Jeong, Daehan Kwon, Jieun Jang, Il Park, Junhyun Chun, Joohwan Cho

SK hynix, Icheon, Korea

With advances in deep-neural-network applications the increasingly large data movement through memory channels is becoming inevitable: specifically, RNN and MLP applications are memory bound and the memory is the performance bottleneck [1]. DRAM featuring processing in memory (PIM) significantly reduces data movement [1,2,3,4], and the system performance is enhanced by the large internal parallel bank bandwidth. Among DRAM-based PIM proposals, [3] is near commercialization, but the required HBM technology may prevent it from being applied to other applications due to its high cost [5]. In this situation, an accelerator-in-memory (AiM) based on GDDR6 may be applicable: it has a relatively low-cost, is compatible with GDDR6 interface, and is designed to accelerate deep-learning (DL) applications. AiM offers a peak throughput of 1TFLOPS with processing units (PUs) with a speed of 1GHz utilizing the characteristics of GDDR6 with a speed of 16Gb/s. It can also support many applications as it has various activation functions. This paper first looks at the AiM architecture and the supported command set for DL operations. Next, the DL operations in the PU and supported activation functions are described. Finally, we present evaluation results of DL behavior of AiM at the package and the system level.

Figure 11.1.1 shows the architecture of the 4Gb AiM. Compared to GDDR6, AiM has PUs for DL operation near the DRAM cells and a 2KB global buffer (GB) for temporary data storage. There are 16 PUs, one for each of the 16 banks; they receive data from DRAM cells and the GB to perform DL operations. The PU operates at 1GHz, which is $t_{\text{CCDS}}(2t_{\text{CK}})$ of GDDR6, and one PU has a throughput of 32GFLOPS. The top of Fig. 11.1.2 is the new command (CMD) set for the DL operation. AiM uses the same CMD set as GDDR6, but has additional CMDs for DL operations. Because AiM performs normal or DL operation based on CMD, long latency required due to mode change is unnecessary, and two operations can be quickly switched via CMD. So, it is possible to perform a DL operation during normal read/write operation. The PU performs multiply-accumulate (MAC) operation that performs matrix product of weights and vectors, element-wise multiplication operation that multiplies weights and vectors respectively, and activation functions that increase DL accuracy through non-linear functions. For MAC operations, it can be performed by receiving weights and vectors from bank and GB, respectively, or by receiving weights from an even bank and receiving vectors from an odd bank without GB. In the former case, 16, 4, or 1 PUs can operate, and in the latter case, 8, 4, or 1 PUs can operate. In addition, it is possible to perform copy operation using GB, which copies 2KB of data corresponding to one row to another, and write bias operation, which adds bias to the DL result.

When MAC CMD is applied once, 256b weights and vectors are output from each column of the bank and GB, and matrix multiplication operation is performed in the MAC circuit of the PU. Figure 11.1.3 shows the structure of the MAC circuit of the PU located in one bank. The MAC circuit consists of 16 multipliers that multiply weights and vectors, an adder tree that adds 16 multiplier outputs into one, and an accumulator. Weights and vectors entering the multiplier use Bfloat16 (BF16) format which is more suitable than FP16 format in the DL operation [6]. BF16 consists of 1b sign (S), 8b exponent (EX), and 7b mantissa (MA). The 7b MA in which hidden bit is omitted becomes 8b by adding a hidden bit in the internal circuit. Two 8b MA are multiplied by multiplier to 16b, and then the fractional part of MA is extended to 23b in a shift circuit to improve accuracy. As the number of bits of MA increases, from the adder tree, the calculation is extended to the FP32 system, not the BF16. Adding 16 multiplier outputs into one requires a floating-point add operation for every 2 outputs. When performing floating-point add operation, it goes through a series of processes of EX comparison, shifting the digits of the MA, and add of the MA operations. Performing these processes in each of the 4 stages of the adder tree consumes too much power, computation time, and area. In order to solve these problems, as shown in Fig. 11.1.3 below, a bank-wide MA shift (BWMS) scheme was proposed that compares 16 EXs from the output of the multipliers to find the maximum EX and completes the shift of all MA at once. Within 16 shifters, each EX is subtracted from the max EX to obtain the differences, and each MA is shifted as much as the difference. In the 4 stages of the adder tree, only simple MA addition is performed

in the MA add unit (MAU) without the EX comparison and MA shift operations. The BWMS scheme reduces power consumption, computation time, and area of the adder tree by 66%, 52%, and 75% respectively, compared to the use of floating-point adders for each addition operation. The structure allowed a large amount of space to be filled with the reservoir capacitor as the PU became smaller. Owing to reservoir capacitors, 16 bank parallel active operations by ACT16 CMD are possible and this CMD can be given without the constraint of t_{AW} , a specification that limits active operation in DRAM. One PU containing the reservoir capacitor has an area of 0.19mm².

The results calculated in the multipliers and the adder tree are added cumulatively in the accumulator. Accumulation occurs as many times as MAC CMD is given, as shown in Fig. 11.1.4, and MAC CMD can be continuously applied at t_{CCDS} intervals as many as 64 times, the number of column addresses, while one word line is activated. Another active command is required for accumulation exceeding 64 MAC CMDs. Since the accumulation requires the completion of one action before the next data enters, it is completed within 1ns. As shown at the bottom of Fig. 11.1.4, if the vector size exceeds 2KB and there are two groups of weights to be multiplied by the vector, a total of 4 vector write operations may be required. Since vector write requires a large amount of data movement through the memory channel, it can cause large current consumption. As can be seen from the MAC structure in Fig. 11.1.3, it has two selectable latches to store the accumulation result, so that the matrix multiplication can be performed by writing the vector only twice.

As shown in Fig. 11.1.5, AiM supports various activation functions (AFs). Sigmoid, hyperbolic tangent, GELU, ReLU, and leaky ReLU are supported by default, and arbitrary AFs can be used by programming. In the case of ReLU and Leaky ReLU, the result values are processed in the internal circuits using MAC results. For sigmoid, hyperbolic tangent, and GELU, the result value $f(x)$ of each function is written on the reserved row of each bank, and when input x is received, it is used as a lookup table (LUT) to output the result. The MAC result is decoded to be the column address corresponding to x , the input of the LUT. In addition to these three AFs, it is also possible to use any AF by programming it into the AF word line. Since the number of inputs in the LUT is limited to 512 and the number of MAC results exceeds that, it is necessary to obtain $f(k)$ for any value of k that does not belong to 512 units. So, a value adjacent to k of 512 inputs is entered into input, $f(A)$ and $f(A+1)$ are output from the LUT, and two values are linearly interpolated to obtain $f(k)$. The linear interpolation allows more accurate function values to be output in a limited number of inputs.

Figure 11.1.6 shows the feature summary and the measurement results. The upper-left Shmoo plot shows that 16Gb/s is reached at 1.25V in DL operation. An FPGA-based platform was developed to estimate AiM system-level performance improvement; the memory interface runs at 2Gb/s/pin, due to FPGA IO limitations. The system-level performance estimation shows up to a 10× speed improvement, compared to the GPU system with HBM2, by the exploited bank-level parallelism. A flat-lid package is used to mitigate the temperature increase. As shown in the lower left of Fig. 11.1.6, the thermal resistance of the flat-lid package is 89% less than a normal over-mold package. Figure 11.1.7 shows the full chip micrograph, a picture of the flat lid package, and the emulation system including the FPGA.

References:

- [1] H. Shin et al., "McDRAM: Low Latency and Energy-Efficient Matrix Computations in DRAM," *IEEE TCAD*, vol. 37, no. 11, pp. 2613-2622, Nov. 2018.
- [2] F. Devaux, "The true Processing In Memory accelerator," *IEEE Hot Chips Symp.*, 2019.
- [3] Y.-C. Kwon et al., "25.4 A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications," *ISSCC*, pp. 350-352, 2021.
- [4] M. He et al., "Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning," *IEEE/ACM MICRO*, pp. 372-385, 2020.
- [5] T. M. Hollis et al., "25.3 An 8Gb GDDR6X DRAM Achieving 22Gb/s/pin with Single-Ended PAM4 Signaling," *ISSCC*, pp. 348-350, 2021.
- [6] D. Kalamkar, et al., "A study of BFloat16 for deep learning training," *arXiv (pre-print)*, id. 1905.12322, 2019.
- [7] S. Lee et al., "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product," *ACM/IEEE ISCA*, pp. 43-56, 2021.

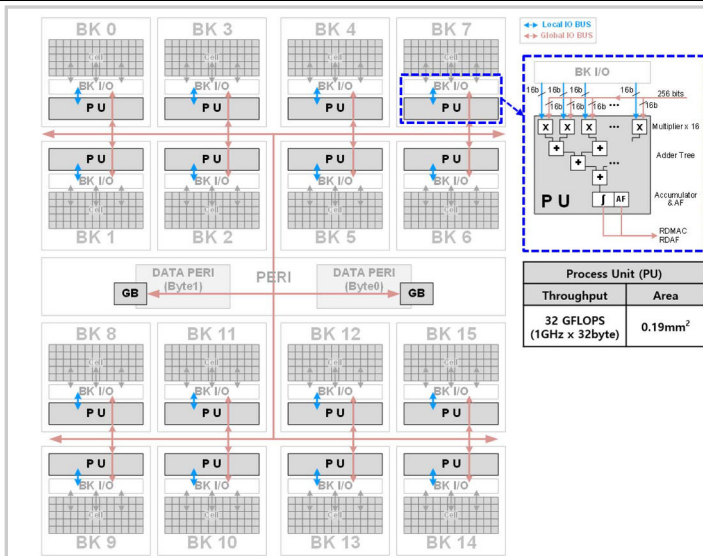


Figure 11.1.1: Accelerator-in-memory (AiM) architecture with 16 processing units (PUs) for deep-learning operations near DRAM cells and a 2KB global buffer (GB) for temporary data storage.

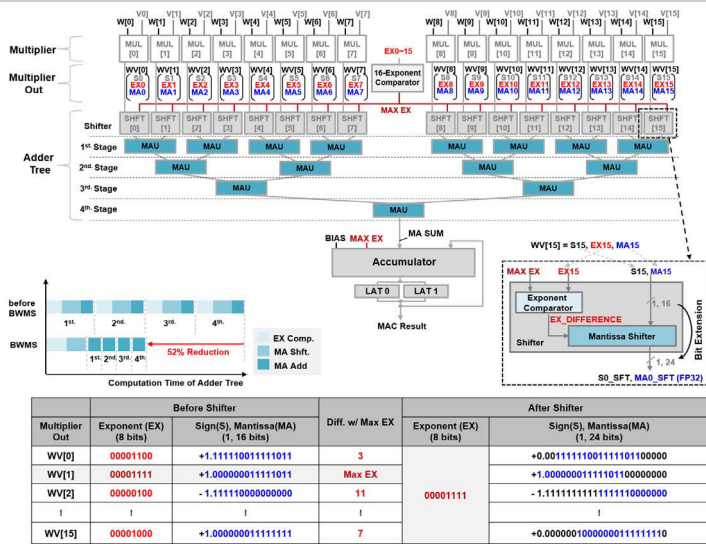


Figure 11.1.3: MAC architecture using the bank-wide mantissa shift (BWMS) scheme that compares 16 exponents (EXs) from the output of the multipliers to find the maximum EX and completes the shift of 16 mantissa at once.

Type	CMD	Description
Bank Activation	ACTA, ACT16	Activate four/sixteen banks in parallel
	ACTAF4, ACTAF16	Activate rows storing activation function LUTs in four/sixteen banks in parallel
Compute	MACSB, MACAB, MACAB	Perform MAC in one/four/sixteen banks in parallel
	AF	Compute activation function in all banks
	EWMUL	Perform element-wise multiplication
	RDCP	Copy data from a bank to Global Buffer
Data Movement	WRCP	Copy data from Global Buffer to a bank
	WRGB	Write to Global Buffer
	RDMAC	Read from MAC result register
	RDAF	Read from activation function result register
	WRBIAS	Write bias to MAC result register
	WRBK	Write to all activated banks in parallel

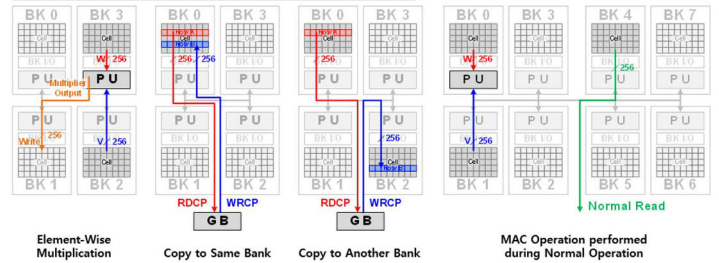
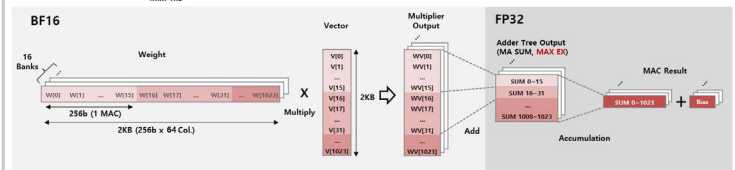
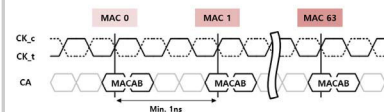


Figure 11.1.2: Newly defined command set for deep-learning operations and data for movement.

Accumulation Operation



Multi-Latch Operation

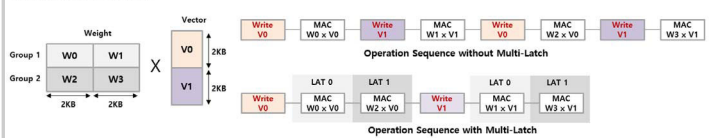


Figure 11.1.4: Accumulation operation that adds cumulatively the results calculated in the multipliers and the adder tree at 1ns intervals (upper). Multi-latch operation that reduces the number of vector writes by selectively storing accumulation results (lower).

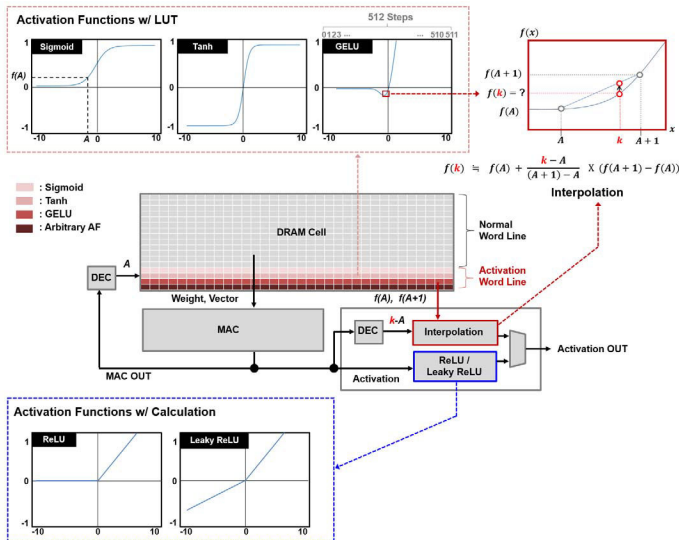
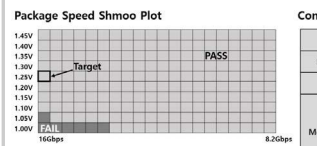
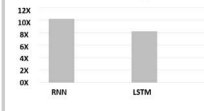


Figure 11.1.5: Activation functions (AFs) that obtain result values through lookup tables (LUTs) or calculations, and linear interpolation to increase the accuracy of AFs that use LUTs.



System Performance Improvements



Thermal Resistance in Two Package Types

DOE	Over-Mold PKG	Flat LID PKG
Pkg Type	EMC	Flat LID
ΘJA (Ambient)	25.9°C/W	25.2°C/W
ΘJB (Board)	9.6°C/W	8.9°C/W
ΘJC (Case of PKG)	4.6°C/W	0.5°C/W

Comparison

	[1]	[2]	[3],[7]	This work
DRAM Type	LPDDR4	DDR4	HBM2	GDDR8
Process	20 nm	2x nm	20 nm	1y nm
Memory Density	8GB/chip (8H 8Gb mono die)	8GB/DIMM	6GB/cube (Buffer die + 4H 4Gb core-die with PCU + 4H 8Gb core-die)	8Gb/chip (4Gb DDP)
Data Rate	3.2Gbps	2.4Gbps	2.4Gbps	16Gbps
Bandwidth	25.6GB/s per chip	19.2GB/s per DIMM	307GB/s per cube	64GB/s per chip
# of Channel	1 per chip	16 per DIMM	8 per cube	2 per chip
# of Processing Unit (PU)	2048 per chip (250 per die)	128 per DIMM (8 per chip)	128 per cube (32 per core-die)	32 per chip (16 per die)
Processing Operation Speed	250MHz	500MHz	300MHz	1GHz
1 PU Throughput	2 GOPS (250MHz x 8Byte)	4 GOPS (500MHz x 8Byte)	9.6 GFLOPS (300MHz x 32Byte)	32 GFLOPS (1GHz x 32Byte)
Total Throughput (1 PU Throughput x # of PU)	0.5 TOPS per chip (2 GOPS x 256)	0.5 TOPS per DIMM (4 GOPS x 128)	1.2 TFLOPS per cube (9.6 GFLOPS x 128)	1 TFLOPS per chip (32 GFLOPS x 32)
Operation precision	INT8	INT8	FP16	BF16
Supported Activation Functions	-	-	ReLU	Sigmoid, Tanh, GELU, ReLU, Leaky ReLU, and Arbitrary AF

Figure 11.1.6: Package speed Shmoo plot, system performance improvements, thermal resistance in two package types, and comparison.

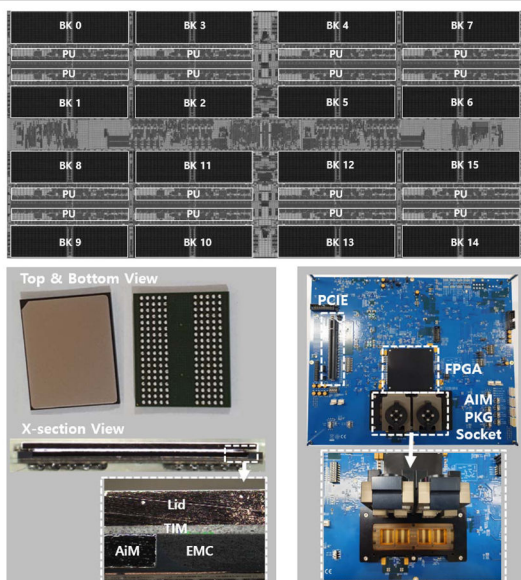


Figure 11.1.7: AiM die micrograph, flat-lid package, and the emulation system.